

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

VALUTAZIONE DELL'EFFETTO DEL CONTESTO DI TEST SUL REFACTORING GENERATO DA LLM: UNO STUDIO SU APACHE AVRO
 Matteo Lagioia
Ingegneria del Software II — A.A. 2024/2025 — progetto individuale (Falessi). Apache Avro — modulo lang/java/avro. Classe target: org.apache.avro.Schema. Baseline C0: release-1.5.4. Repository: <https://github.com/StitchML/avro>. CI: <https://github.com/StitchML/avro/actions>. SonarCloud: https://sonarcloud.io/project/overview?id=StitchML_avro. Data: 24 aprile 2026.

Abstract—Questo lavoro studia se fornire a un Large Language Model delle suite di test strutturate come parte del contesto di refactoring ne migliori in modo misurabile il comportamento. Su un hotspot di Apache Avro (org.apache.avro.Schema, release 1.5.4), abbiamo costruito un dataset class-level auditabile su 43 release (3 183 righe, 55 feature organizzate secondo la convenzione Asterisco con prefissi max..., ever..., Total..., 14,5% di positivi, Linkage Rate commit-level 90,13% e ticket-level 40,1%) usando SZZ leggero con fallback Proportion (P medio calibrato 0,9805, applicato al 23,3% dei ticket con fix commit privi di affectedVersions). Abbiamo valutato in walk-forward a 13 fold tre classificatori Weka (Naive Bayes, Random Forest, IBk), eletto il winner secondo Kappa-then-AUC, e stimato l'effetto di un refactoring ideale azzerando le quattordici colonne azionabili del dataset (sette istantanee più sette storiche). Abbiamo poi prodotto quattro varianti di refactoring della classe tramite GPT-5.4 e Claude Opus 4.6 su quattro livelli di contesto (C1 solo sorgente; C2 + black-box; C3 + control flow; C4 + mutanti sopravvissuti) e valutato le varianti sulla stessa suite di 90 test (82 unit + 8 IT) scritta su C0. La variante C2 è quella che massimizza la qualità percepita del refactoring preservando il comportamento; l'aggiunta di control flow (C3) conferma ma non amplifica il beneficio, e l'aggiunta di mutanti (C4) lo degrada. Il risultato è coerente con l'ipotesi che la specifica behavior-level sia la granularità giusta per il ragionamento di un LLM.

Index Terms—Apache Avro, defect prediction, large language models, maintainability, mutation testing, proportion algorithm, refactoring, SZZ, walk-forward validation, what-if analysis.

I. Introduzione

Questo lavoro si colloca nel percorso didattico di Software Engineering e chiude la sequenza di quattro milestone iniziata con la costruzione del dataset class-level su Apache Avro (M1), proseguita con la classificazione supervisionata e l'analisi what-if su feature azionabili (M2), e continuata con la sperimentazione di una suite di test multilivello accompagnata da un protocollo di refactoring guidato da LLM su una singola classe hotspot (M3). L'obiettivo di questa relazione è fornire una lettura unitaria del percorso e, soprattutto, rispondere a una domanda di ricerca precisa: dare a un Large Language Model delle suite di test strutturate come parte del contesto ne migliora in modo misurabile la qualità del refactoring rispetto al caso in cui gli si fornisca soltanto il sorgente della classe da modificare?

La scelta del progetto Apache Avro risponde a tre requisiti sperimentali non negoziabili. Primo, un progetto open-source maturo con più di dieci anni di storia, test originali ben strutturati che funzionano da oracolo indipendente, e un sistema di build Maven stabile. Secondo, un dominio applicativo, quello della serializzazione binaria con supporto a schemi dinamici, che mette in primo piano un oggetto centrale: la classe Schema, che incrocia in modo naturale categorie di test eterogenee (factory method, parser, equality, validazione, union, record, enum). Terzo, una quantità di release sufficiente a supportare una validazione temporale walk-forward coerente.

La risposta sintetica alla domanda di ricerca è affermativa ma con una qualificazione importante. Fornire al modello la suite black-box e la suite control-flow come contesto aggiuntivo produce varianti di refactoring più stabili e più vicine all'ottimo locale in termini di complessità, mentre aggiungere in input anche l'elenco dei mutanti sopravvissuti introduce rumore e non porta a un miglioramento ulteriore. Il mutation score sulla classe target oscilla tra il 45 per cento e il 48 per cento a seconda della variante, con una test-strength stabile tra il 78 e il 79 per cento, e tutte le varianti superano l'intera suite di novanta test (ottantadue unit più otto di integrazione) progettata sulla versione baseline.

Questa relazione è organizzata in otto capitoli. Il capitolo due documenta la selezione della classe target e della release di riferimento, il capitolo tre descrive la costruzione del dataset e le decisioni metodologiche prese nel labeling, il capitolo quattro riassume l'analisi di machine learning e lo studio what-if su feature azionabili, il capitolo cinque descrive la metodologia adottata in M3 e il protocollo di refactoring LLM, il capitolo sei presenta e discute i risultati, il capitolo sette espone i threats to validity, e il capitolo otto conclude con una lettura aggregata e le direzioni di lavoro futuro. Figure, tabelle, prompt e listati di codice, quando serve riportarli per trasparenza, sono confinati in appendice e richiamati dal testo per riferimento.

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

II. Selezione della classe target

La scelta di `org.apache.avro.Schema` come unica classe su cui condurre la sperimentazione M3 nasce dall'intersezione tra criteri quantitativi, derivati dal dataset class-level prodotto in M1, e criteri qualitativi legati alla superficie pubblica della classe. Dal punto di vista quantitativo, il ranking per numero di smell distinti per release (NSmells), aggregazione del conteggio smell totale (CodeSmells) e densità normalizzata (SmellDensity) pone Schema tra gli hotspot più persistenti del modulo Java di Avro. La classe si presenta con valori elevati di NSmells in quasi tutte le release osservate e compare con ricorrenza nel sottoinsieme di istanze etichettate come buggy dal labeling SZZ+Proportion descritto al capitolo successivo. Il segnale è stabile e non dipende dalla particolare release scelta come baseline, il che riduce il rischio che la selezione sia guidata da un picco episodico. Dal punto di vista qualitativo, la scelta di Schema è dettata dalla ricchezza della sua interfaccia pubblica. La classe espone factory method per tutti i tipi Avro (record, enum, union, fixed, array, map, map di record, tipi primitivi), un meccanismo di parsing testuale (parse), un sistema di equality e toString strutturale, la gestione dei campi di un record via setFields, e una serie di vincoli interni (nomi univoci, validazione dei tipi, gestione di null). Questa varietà è ideale per costruire suite di test eterogenee: una Category Partition ragionata sulla classe aveva già prodotto, prima di M3, un corpus ampio di casi nominali, limite ed errore, pronto per essere tradotto in classi JUnit. Il costo di entrata sulla black-box è quindi ridotto rispetto alle altre candidate Avro, lasciando più tempo per le tecniche avanzate (control flow, mutation-guided, coverage-guided).

La versione baseline scelta è release-1.5.4. Due motivazioni giustificano questa decisione. La prima è la coerenza col perimetro storico del dataset M1/M2: il labeling di bugginess è costruito sull'intera timeline Avro osservabile (43 release, da release-1.0.0 a release-1.11.5) mentre l'emissione delle righe è limitata al primo 33% delle release in ordine cronologico, cioè la finestra release-1.0.0 ... release-1.5.4. Scegliere release-1.5.4 come C0 significa scegliere l'estremo destro di questa finestra: la classe Schema viene riscritta esattamente nella release più recente in cui figura nel dataset, il che evita la distorsione che deriverebbe dal portare Schema a una release successiva ai dati utilizzati per classificarla. La seconda è operativa: la versione 1.5.4 contiene già il modulo Maven `lang/java/avro` nella forma che il progetto assume fino alla 1.8, con la classe `Schema.java` nel path standard `lang/java/avro/src/main/java/org/apache/avro/Schema.java`. La toolchain conservativa, Temurin Java 8 con Maven 3.6, è sufficiente alla compilazione e al test del modulo senza forzare aggiornamenti invasivi sul resto del build. Un piano B era stato esplicitamente previsto nella pianificazione iniziale, consistente nello spostare C0 a release-1.8.0 mantenendo stessa classe e stesso path, ma non è risultato necessario.

Una precisazione va fatta sul criterio di unicità del target. La specifica consente di scegliere più di una classe, ma abbiamo optato consapevolmente per una sola. La ragione è che una sperimentazione a classe singola permette di spingere al limite la profondità dell'analisi (sei livelli di testing più integrazione, quattro varianti di refactoring LLM, coverage e mutation per ciascuna) senza diluire il budget di tempo su più target. Il rischio di sovraspecializzazione del risultato sul caso Schema è reale ed è discusso apertamente al capitolo sette. Il valore atteso di una scelta a classe singola ben condotta è comunque superiore al valore atteso di uno studio superficiale su tre classi, e questa è la traiettoria che abbiamo seguito.

III. Costruzione del dataset (M1)

Il dataset class-level che alimenta tanto la milestone di machine learning quanto la scelta della classe target di M3 è stato prodotto dal pipeline MantiMetrics, un'applicazione Java 17 costruita in modo deliberatamente modulare per evitare ogni tentazione di god class. L'intera estrazione è orchestrata da `ApplicationBootstrap` nel package `com.mantimetrics.app` e si distribuisce su undici package a responsabilità singola, ciascuno con un ruolo chiaramente distinguibile: risoluzione della configurazione e input CLI (cli, config), pianificazione delle release (analysis), interazione con GitHub e Jira (git, jira), costruzione dell'oracolo storico dei bug (labeling), parsing e metriche statiche (parser, metrics, clone), accumulo delle metriche di processo (history), modellazione delle righe (model), scrittura delle uscite (csv, dataset) e audit di milestone (audit). Questa scelta architetturale è documentata anche nel file `docs/milestone1-architecture.md` della repository MantiMetrics ed è importante da dichiarare perché implica che ogni passo del labeling è tracciabile a un componente nominato e testato in isolamento, e non a uno script monolitico.

La configurazione di progetto risiede in `src/main/resources/projects-config.json`: per Avro fissa `owner=apache`, `name=Avro`, `repoUrl=https://github.com/apache/avro.git`, `jiraKey=AVRO`, `percentage=33`. Quest'ultimo è il parametro di snoring descritto a breve. Le credenziali GitHub e Jira sono isolate in `config/github.local.properties` e `config/jira.local.properties`, file non versionati. La query JQL usata per interrogare Jira è `project = AVRO AND issuetype = Bug AND status in (Closed, Resolved) AND resolution = Fixed`, impostata in `src/main/resources/jira.properties`.

Il pipeline, una volta avviato, si articola in sei fasi coerenti con i package citati. La prima fase, svolta da `ProjectReleasePlanner`, costruisce la timeline cronologica delle release comuni a Git e a Jira normalizzando i tag (`JiraClient::normalize`) e intersecando i tag del repository con le versioni pubblicate sul progetto Jira. Per Avro il risultato è una timeline di 43 release complessive, da

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

release-1.0.0 (2009) a release-1.11.5 (2025), dettagliata nel file di audit output/avro_dataset_class_artifacts/avro_release_timeline.csv. La seconda fase, sempre in ProjectReleasePlanner tramite ReleaseSelector::selectFirstPercent, applica la politica di snoring nella formulazione richiesta dalla traccia del corso: si scarta l'ultimo 66% della timeline (le release troppo recenti per avere bug già scoperti, triati, fissati e tracciati a un commit) e si emettono righe solo sul primo 34% in ordine cronologico, pur continuando a usare i ticket dell'intero storico per labellare quelle release. Il parametro concreto nel file src/main/resources/projects-config.json è percentage=33 (ReleaseSelector::selectFirstPercent emette il primo 33% del timeline, che è l'implementazione più conservativa del requisito «primo 34%» e la più vicina al dato reale di cut-off osservato). Nel nostro estratto questo si traduce in 14 release selezionate su 43 (32,56%), da release-1.0.0 a release-1.5.4 inclusa, che è esattamente la baseline C0 scelta per la milestone 3; il file di audit milestone1-audit.json dichiara il criterio testualmente: «Rows are emitted only for the oldest release window; the full timeline is still used for labels». Questo è il punto cruciale da sottolineare: le 29 release scartate dall'emissione delle righe partecipano comunque al labeling, perché sono la finestra di osservazione postuma che consente di riconoscere se una classe “giovane” sia stata in realtà già ritoccata da un fix successivo. In altri termini, la traccia richiede di ignorare l'ultimo 66% per l'emissione, ma non per l'oracolo: l'oracolo vive sull'intera timeline.

La terza fase è la raccolta dei ticket bug e della storia Git. JiraClient (package jira) paginatamente legge i ticket resolved+fixed; per Avro il conteggio totale registrato in milestone1-audit.json è di 1 101 ticket. Per ciascuna release selezionata ReleaseExecutionService scarica lo snapshot sorgenti tramite GitHubRepositoryClient e costruisce un ReleaseSnapshot che include la ReleaseCommitData: per ogni file modificato nel range di commit della release, memorizza l'elenco di chiavi Jira citate nel messaggio (pattern AVRO-\d+) e le metriche di processo aggregabili (touches, issue-linked touches, authors, added/deleted lines, churn). Su questa fase il pipeline calcola due indicatori distinti di qualità del linkage Jira ↔ Git, ed è importante non confonderli. Il primo, scritto nel campo linkageRate dell'audit JSON e calcolato da ProjectProcessor::computeLinkageRate, è il Linkage Rate a livello di commit: vale la proporzione di commit che toccano almeno un file Java e contengono almeno una chiave Jira nel messaggio, aggregata su tutte le 43 release della timeline. Per Avro questo valore è 0,9013, cioè 90,13%: significa che il 90% dei commit Java porta nel messaggio un riferimento esplicito a un ticket. Il valore è abbondantemente sopra la soglia di 0,20 indicata nello stesso campo linkageRateNote come limite di affidabilità: più il rate è alto, più la convenzione di commit message del progetto è disciplinata, più stretta è la corrispondenza tra fix commit e fault-introducing changes nello SZZ. Il secondo indicatore, di tipo ticket-level, si ricava per derivazione dai contatori totalResolvedTickets (1 101) e ticketsWithFixCommit (442) ed è pari a $442/1\ 101 \approx 40,1\%$: misura quanti dei ticket dichiarati Resolved+Fixed dalla query JQL trovano effettivamente almeno un fix commit nella timeline. La differenza tra il 90,13% e il 40,1% è attesa ed è una proprietà strutturale di Avro: i commit Java sono molto più numerosi dei ticket fixed (ogni ticket genera in media molti commit, e molti commit Java sono refactoring o piccole correzioni che non chiudono un bug ticket); il primo indicatore quantifica la disciplina di commit, il secondo quantifica la copertura dell'oracolo. Entrambi vanno interpretati: 0,9013 ci rassicura che lo SZZ ha un Git log informativo su cui lavorare, mentre 0,401 ci ricorda che il 60% dei ticket fixed non lascia tracce aggangiabili e quindi non contribuisce all'oracolo. Questa quota mancante è un limite di recall, non di precision: la proporzione di righe Buggy=yes (14,5%) è verosimilmente un lower bound dell'effettiva fault-proneness.

La quarta fase è la costruzione dell'oracolo storico di bugginess. È affidata a HistoricalBugLabelIndexBuilder e segue una politica descritta nel campo labeling.strategy dell'audit JSON come proportion-fallback. La politica è gerarchica. Per ogni ticket si scandisce l'intera timeline per individuare la release di fix FV, definita come la prima release il cui commit range cita la chiave del ticket su uno o più file; tra questi file vengono raccolti i path toccati. Stabilito FV, l'algoritmo cerca di inferire la release di introduzione del difetto (Injected Version, IV). Se il ticket ha affectedVersions valorizzate e almeno una di esse è antecedente a FV, IV è la più antica tra quelle candidate. In assenza di affected version consistenti, si applica la formula di Proportion classica $IV = FV - (FV - OV) \cdot P$, dove OV è l'Opening Version inferita dalla data di creazione del ticket tramite ReleaseTimeline::findOpeningVersionIndex e P è la proporzione media calibrata dalle sole issue con affected version note. Per Avro la pipeline ha osservato 1 101 ticket, 442 dei quali correlati a un fix commit nella timeline; di questi 442, 287 hanno affected version usabili (e quindi IV viene letto direttamente da Jira), 103 cadono nel fallback Proportion, e 52 (11,8%) producono un range [IV, FV] degenerare — perché tutte le affected version sono successive a FV e contemporaneamente l'Opening Version risulta $\geq FV$ — e vengono esclusi dall'oracolo dal continue su injectedIndex $\geq$ fixIndex in HistoricalBugLabelIndexBuilder. Il valore calibrato per Avro è $P = 0,9805$, un P vicino a uno che rappresenta una calibrazione di fatto pessimista: significa che, in media, la classe risulta buggy su quasi tutta la finestra tra OV e FV. Quando non esiste alcun ticket con affected version valide per calibrare P, il codice fa default a $P = 1,0$, equivalente a $IV = OV$, esplicitato come fallback

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

conservativo in `HistoricalBugLabelIndexBuilder::computeProportionP`. Ottenuti FV e IV, per ciascun indice di release nell'intervallo [IV, FV) vengono marcati come buggy tutti i path toccati dal ticket.

La quinta fase è l'estrazione delle metriche statiche e degli smell. `CodeParser` usa `JavaParser` per navigare l'albero sorgente di ciascuna release scaricata, `MetricsCalculator` produce le quattordici metriche statiche per release: LOC, StmtCount, complessità ciclomatica e cognitiva, la profondità massima di nesting (`MaxNestingDepth`), e le nove metriche di Halstead (`DistinctOperators`, `DistinctOperands`, `TotalOperators`, `TotalOperands`, `Vocabulary`, `Length`, `Volume`, `Difficulty`, `Effort`). I quattro smell flag — `isLongMethod`, `isGodClass`, `isFeatureEnvy`, `isDuplicatedCode` — sono computati da detector coesi del package metrics (e per la duplicazione dal `CloneDetector` del package clone) secondo le definizioni classiche documentate in `docs/metrics.md`: Long Method su un threshold di LOC del metodo, God Class su una combinazione di dimensione e coupling della classe, Feature Envy su un'euristica di accesso cross-class, Duplicated Code su un rilevatore a token. La traccia del corso richiede che `SmellDensity` sia calcolata mediante `SonarCloud`; in questa implementazione la stessa definizione operativa (numero di smell attivi diviso LOC) è stata riprodotta in-house a partire dai quattro detector suddetti, per due ragioni: garantire la riproducibilità offline del pipeline (`SonarCloud` richiede autenticazione OAuth e una quota free limitata, insufficiente a reistanziare la scansione sulle 43 release della timeline) e mantenere una semantica dei flag di smell coerente con il detector della duplicazione a token, che è ciò che pilota la feature azionabile `isDuplicatedCode` del what-if. La deviazione rispetto alla traccia è dichiarata esplicitamente nei threats to construct validity al capitolo VII. A livello di classe-release, `NSmells` conta il numero di smell istantanei attivi in quella release, `SmellDensity` è definito come `NSmells / LOC`, e `CodeSmells` è la somma cumulativa degli smell sulla stessa classe nelle release precedenti, aggiornata progressivamente da `RowHistoryStore` (package history).

La sesta fase è la storicizzazione delle metriche e la scrittura del dataset. `DatasetRowEnricher` legge da `RowHistoryStore` lo stato `historyState` della classe corrente — uno stato accumulato strettamente in ordine cronologico sulla timeline — e popola ciascuna riga con tre famiglie di feature aggiuntive che insieme realizzano in modo uniforme la convenzione Asterisco richiesta dalla traccia. La traccia recita che per ogni metrica vanno esposte due colonne, una relativa alla sola release corrente e una all'intero storico della classe («concetto dell'asterisco»). Nel nostro dataset questa duplicazione è realizzata con tre convenzioni di prefisso diverse, scelte per coerenza semantica con il tipo della metrica. Per le metriche statiche additive (LOC, StmtCount, Cyclomatic, Cognitive, le nove di Halstead, `MaxNestingDepth`) il valore «con asterisco» è il massimo storico osservato sulla classe ed è esposto come `maxLOC`, `maxStmtCount`, `maxCyclomatic`, `maxCognitive`, `maxDistinctOperators`, `maxDistinctOperands`, `maxTotalOperators`, `maxTotalOperands`, `maxVocabulary`, `maxLength`, `maxVolume`, `maxDifficulty`, `maxEffort`, `maxNestingDepth`; lo stesso vale per `maxNSmells`, `maxCodeSmells` e `maxSmellDensity`. La scelta del massimo (e non della somma) è semanticamente più informativa per metriche statiche: la somma di LOC su tutte le release sarebbe puramente proporzionale all'età della classe e ridondante con `AgeInReleases`, mentre il massimo cattura il «picco di complessità» raggiunto, che è il segnale di rischio che vogliamo. Per i quattro flag binari di smell la versione «con asterisco» è l'OR storico, esposta come `everLongMethod`, `everGodClass`, `everFeatureEnvy`, `everDuplicatedCode`: vale true se lo smell è stato attivo in almeno una release passata, e questo è il proxy storico dell'esposizione cumulativa al pattern. Per le metriche di processo additive (touches, issue-linked touches, authors, churn) la versione «con asterisco» è la somma cumulativa, esposta come `TotalTouches`, `TotalIssueTouches`, `TotalAuthors`, `TotalChurn`. In sintesi, il prefisso `max...` cattura il picco storico, `ever...` cattura l'occorrenza booleana storica, `Total...` cattura l'accumulo additivo storico; queste tre convenzioni insieme sono la realizzazione operativa, type-aware, dell'asterisco della traccia. Abbiamo deliberatamente evitato di suffissare le colonne con il carattere * per tre ragioni concrete: il carattere è soggetto a escaping ambiguo in CSV/ARFF, viene usato dall'attribute selection di Weka come placeholder, e renderebbe i nomi colonna non identifier-safe in molte librerie ML. Le tre famiglie sono documentate in `DatasetColumns.java` e in `docs/metrics.md` come equivalenti dell'asterisco. La scrittura finale è eseguita da `CSVWriter` che produce il CSV grezzo, e da `DatasetArtifactService` che invoca `WhatIfDatasetBuilder` per le quattro derivazioni a valle.

Il CSV risultante `output/avro_dataset_class.csv` ha 3 183 righe e sessanta colonne: quattro identificatori (Project, Path, Class, ReleaseId; notare l'assenza di Method, coerente con la granularità class-level scelta), cinquantacinque feature e la label nominale Buggy $\in \{\text{yes}, \text{no}\}$. Le 55 feature si distribuiscono in sette gruppi: (i) quattordici metriche statiche istantanee della release corrente (LOC, StmtCount, Cyclomatic, Cognitive, DistinctOperators, DistinctOperands, TotalOperators, TotalOperands, Vocabulary, Length, Volume, Difficulty, Effort, MaxNestingDepth); (ii) quattro flag di smell istantanei (`isLongMethod`, `isGodClass`, `isFeatureEnvy`, `isDuplicatedCode`); (iii) tre aggregati smell istantanei o cumulativi (`CodeSmells` cumulativo, `NSmells` istantaneo, `SmellDensity`); (iv) quindici metriche statiche `max...` storiche, ovvero il picco osservato della corrispondente metrica istantanea sulla classe (`maxLOC`, `maxStmtCount`, `maxCyclomatic`, `maxCognitive`, `maxNSmells`, `maxDistinctOperators`, `maxDistinctOperands`, `maxTotalOperators`, `maxTotalOperands`, `maxVocabulary`, `maxLength`,

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

maxVolume, maxDifficulty, maxEffort, maxNestingDepth); (v) quattro flag ever... (everLongMethod, everGodClass, everFeatureEnvy, everDuplicatedCode), OR storici dei flag istantanei; (vi) due aggregati smell storici max... (maxCodeSmells, maxSmellDensity); (vii) dieci feature di processo (Touches, TotalTouches, IssueTouches, TotalIssueTouches, Authors, TotalAuthors, AddedLines, DeletedLines, Churn, TotalChurn); (viii) tre feature storiche di label/age (prevCodeSmells, AgeInReleases, prevBuggy). Il conteggio è consistente con quanto riportato dal file milestone1-audit.json (featureCount: 55). Lette nel modo giusto, queste 55 feature compongono la convenzione Asterisco end-to-end: i gruppi (i)+(ii)+(iii) e il sotto-blocco istantaneo del gruppo (vii) (Touches, IssueTouches, Authors, AddedLines, DeletedLines, Churn) sono i valori «della release corrente»; i gruppi (iv)+(v)+(vi) e il sotto-blocco Total... del gruppo (vii) (TotalTouches, TotalIssueTouches, TotalAuthors, TotalChurn) sono i valori «dell'intero storico» della classe; i tre del gruppo (viii) sono memorie temporali di label/età, complementari all'asterisco. Delle 3 183 righe, 462 portano la label Buggy=yes (proporzione di positivi 14,5%) e 2 721 portano Buggy=no; 2 355 righe hanno NSmells > 0 (presenza di almeno uno smell azionabile) e 828 sono smell-free. Questi numeri sono scritti testualmente nei campi buggyRows, cleanRows e smellyRows dell'audit JSON, insieme al linkageRate di 0,9013 e ai contatori di labeling, ed è ciò che rende la milestone 1 auditabile senza dover rieseguire il pipeline.

Dal CSV grezzo WhatIfDatasetBuilder costruisce quattro dataset classifier-ready, tutti prodotti in coppia CSV/ARFF nella cartella output/avro_dataset_class_artifacts/. Il dataset A (3 183 righe) è la proiezione completa su tutte le feature non-identificative, ed è quello su cui viene addestrato il classificatore winner di M2. Il dataset B+ (2 355 righe) è il filtro di A sulle sole classi con almeno uno smell (NSmells > 0), pensato come popolazione “a rischio”. Il dataset B (2 355 righe) è la copia controfattuale di B+ in cui le quattordici colonne azionabili dichiarate in metadata.actionableColumns vengono poste a zero (per i numerici) o a false (per i booleani), simulando l'intervento ipotetico di un refactoring che elimina gli smell identificati. Le quattordici colonne azzerate sono i tre aggregati smell istantanei (CodeSmells, NSmells, SmellDensity), il picco storico maxNSmells, i quattro flag istantanei (isLongMethod, isGodClass, isFeatureEnvy, isDuplicatedCode), i quattro flag storici ever... (everLongMethod, everGodClass, everFeatureEnvy, everDuplicatedCode) e i due aggregati storici maxCodeSmells, maxSmellDensity. L'inclusione delle colonne storiche tra le azionabili è una scelta forte ma coerente con la semantica del what-if: la traccia chiede di stimare l'impatto di un refactoring «ideale», e un refactoring ideale rimuove non solo lo smell istantaneo ma anche la traccia che la classe ne abbia mai sofferto, perché l'oracolo storico è ciò che induce il segnale residuo nelle metriche prev... di future release. Il dataset C (828 righe) è infine il complemento di B+, cioè le classi senza smell istantaneo, usato come popolazione di controllo smell-free. La transizione B+ → B è quella che quantifica l'impatto del refactoring, ed è l'unica transizione in cui due dataset condividono la stessa selezione di righe ma differiscono nei valori delle feature azionabili; i dettagli numerici sono in A.1 e in A.2.2.

L'intero pipeline è riproducibile: il punto di ingresso è ApplicationBootstrap::main, attivato con mvn -q -pl . compile exec:java, e produce il CSV grezzo, i quattro dataset derivati, i file di timeline e smell per ispezione (avro_release_timeline.csv, avro_class_smells.csv, avro_buggy_class_rows.csv) e il file di audit milestone1-audit.json che documenta in forma machine-readable la policy di snoring applicata, la strategia di labeling, la distribuzione di righe buggy/clean/smelly e la copertura del linkage Jira. La convergenza tra audit, timeline e CSV è voluta: una correzione del pipeline viene rilevata automaticamente da qualunque consumatore del dataset, senza dover leggere codice.

IV. Machine learning e analisi what-if (M2)

La milestone di machine learning è realizzata dall'applicazione Milestone2WekaEvaluation, un progetto Java deliberatamente frammentato in slice (validation/, validation/timeseries/, whatif/, reporting/, crossvalidation/) per isolare le responsabilità di caricamento dati, selezione della strategia di validazione, valutazione dei modelli, analisi controfattuale e scrittura dei report. Il punto di ingresso AnalysisApplication apre il bundle di output output/, AnalysisRunner enumera i dataset prodotti in M1 e delega a DatasetAnalyzer, che a sua volta invoca ModelEvaluator per la fase di validazione e WhatIfAnalyzer per lo studio controfattuale. Il writer finale DatasetReportPublisher produce i cinque CSV di interesse (results.csv, fold_metrics.csv, milestone2_summary.csv, feature_correlations.csv, what_if_summary.csv) più le due figure riassuntive output/charts/avro_dataset_class_bar.png e output/charts/avro_dataset_class_box.png. La riproducibilità è garantita dal profilo di lancio run-analysis.cmd, che fissa seed, strategia, attributo temporale, classe positiva e policy SMOTE/what-if; l'esecuzione registrata come RunId=20260422-181100 è quella da cui provengono tutti i numeri riportati in questo capitolo e nell'appendice A.2.

La strategia di validazione adottata è walk-forward sull'attributo ReleaseId, scelta obbligata per un esperimento di defect prediction in cui il leakage temporale deve essere escluso per costruzione. L'estrattore dei periodi TimeSeriesWindowGenerator produce 13 fold consecutivi con finestra di training espansiva: il primo fold addestra su una sola release (187 istanze) e valuta sulla successiva, il fold finale addestra su 12 release (2 873 istanze) e valuta su release-1.5.4 (310 istanze). Un dettaglio

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

importante da chiarire subito è che ReleaseId ordina le release per data di rilascio secondo Jira e non per semver: la sequenza dei test set risulta infatti release-1.3.2 → 1.3.1 → 1.3.0 → 1.2.0 → 1.1.0 → 1.0.0 → 1.3.3 → 1.4.0 → 1.4.1 → 1.5.0 → 1.5.1 → 1.5.2 → 1.5.3 → 1.5.4, come visibile nella colonna TestWindow di fold_metrics.csv. Questo ordine cronologico non monotono rispetto alla numerazione è conseguenza del branching di manutenzione di Avro (la 1.3.1 e la 1.3.0 sono patch uscite dopo la 1.3.2), e condiziona in modo decisivo la distribuzione di positivi tra fold.

La combinazione tra labeling Proportion-fallback calibrato a $P = 0,9805$ e distribuzione dei ticket Jira concentrata sui fix delle release più recenti ha un effetto diretto sulla valutazione: i test set dei fold 0 – 7 (release 1.3.1, 1.3.0, 1.2.0, 1.1.0, 1.0.0, 1.3.3, 1.4.0, 1.4.1) contengono zero positivi reali, mentre i test set dei fold 8 – 12 (le cinque release 1.5.0 ... 1.5.4) portano la quasi totalità del carico buggy del dataset. Questo fatto è visibile nelle celle NaN su Precision, Recall, F1 e AUC dei fold 2 – 4 (assenza completa di positivi, sia attesi sia predetti) e in Kappa che degenera a 1,0 quando un classificatore predice correttamente «no» su una popolazione tutta negativa. Le conseguenze sull'aggregazione media per classificatore sono analizzate esplicitamente più avanti. L'attributo sensibile per NPofB20 è LOC, coerente con la lettura canonica della metrica come frazione dei bug individuati ispezionando il venti per cento del codice ordinato per dimensione. Il seed è 42, la classe positiva è yes.

Sulla policy di bilanciamento, la specifica d'esame ammette Undersampling, Oversampling o SMOTE e impone la regola ferrea che il bilanciamento avvenga esclusivamente sul training set. La baseline M2 qui riportata è eseguita senza bilanciamento (colonna Balancing=none in results.csv, parametro --smote=false del profilo run-analysis.cmd). Questa decisione ha due ragioni. La prima è di ordine metodologico: in un esperimento walk-forward con training espansivo il numero di positivi nel training cresce di fold in fold, e un SMOTE fold-wise produrrebbe distribuzioni di minoranza sintetiche di ampiezza variabile che rendono difficile leggere il contributo netto dell'accumulo di dati rispetto al contributo del bilanciamento. La seconda è operativa: il pipeline BalancingStrategyRegistry supporta già i tre schemi sopra citati e sono attivabili via riga di comando per un'analisi di sensibilità futura senza modifiche di codice; la corsa riportata in questo capitolo serve da riferimento «no-rebalancing» rispetto al quale misurare eventuali incrementi di Recall in analisi successive. Il bilanciamento, se attivato, viene applicato dal fold generator TimeSeriesWindowGenerator tramite FilteredClassifier esclusivamente sulle istanze di training di ciascun fold, lasciando intatto il test set — come richiesto dalla spec.

Un chiarimento è inoltre doveroso sulla granularità. La specifica del corso fa riferimento a «metodi buggy» nella formulazione dell'analisi what-if, ma la granularità effettiva del dataset avro_dataset_class.csv è di classe (Granularity=CLASS in results.csv e assenza della colonna Method nel CSV grezzo prodotto da M1). La scelta class-level è stata compiuta in fase di estrazione M1 per contenere il costo computazionale e garantire che il labeling Proportion producesse statistiche stabili (i metodi sarebbero stati troppi e con pochissimi tocchi per unità, rendendo il fit del P meno affidabile). La quantità stimata nel what-if va dunque letta come «istanze di classe buggy evitabili tramite refactoring» e non «metodi»: l'ordine di grandezza è lo stesso ma la traduzione a method-level richiederebbe un secondo pipeline di estrazione. Questa deviazione è esplicitata nel capitolo 7 tra i threats to construct validity.

Prima della valutazione il pipeline esegue un'analisi di correlazione feature-label tramite FeatureCorrelationAnalyzer, che ordina tutte le colonne numeriche per correlazione assoluta con Buggy e la scrive in feature_correlations.csv. La classifica è istruttiva e indica con chiarezza dove il segnale sia più denso. I primi tre driver sono metriche di processo cumulative: TotalAuthors a 0,411, TotalTouches a 0,365, TotalIssueTouches a 0,357, seguite da TotalChurn a 0,264 e dalla coppia IssueTouches/Touches a 0,215/0,213. Le metriche di processo istantanee (Authors, AddedLines, DeletedLines, Churn) seguono tra 0,19 e 0,16. Le metriche statiche di complessità (Cyclomatic, Cognitive, Vocabulary, DistinctOperands, MaxNestingDepth) si addensano in una fascia 0,10 – 0,14, e il cluster degli smell (NSmells, CodeSmells, SmellDensity, isLongMethod, isGodClass) si colloca poco sopra e poco sotto lo 0,10. La lettura aggregata è che, nel dataset Avro class-level, la storia recente del file (touches, authors, churn aggregato) ha un potere predittivo più forte della forma sintattica del codice nel singolo snapshot. Nonostante questo, la feature effettivamente selezionata per lo studio controfattuale è NSmells (rank 20, correlazione 0,101) e non TotalAuthors, perché la traccia del corso impone l'uso di una feature azionabile ed è NSmells l'unica colonna in top-venti il cui azzeramento corrisponde a un'azione di ingegneria praticabile (l'eliminazione degli smell via refactoring); FeatureCorrelationAnalyzer registra infatti la riga SelectedForWhatIf=true sulla sola feature NSmells con reason preferred exam feature 'NSmells'. Questa scelta priva lo studio what-if di una parte del segnale (una feature a correlazione quattro volte maggiore rimane non azionata), ma è coerente con la domanda di ricerca e con il vincolo didattico, e spiega fin da subito perché i delta controfattuali risulteranno modesti in valore assoluto.

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

Tre famiglie di classificatori Weka sono state valutate in parallelo su tutti i 13 fold: Naive Bayes (weka.classifiers.bayes.NaiveBayes), Random Forest (weka.classifiers.trees.RandomForest) e IBk con $k=1$ (weka.classifiers.lazy.IBk). Il risultato, così come registrato in milestone2_summary.csv, è un paesaggio di metriche-winner frammentato: Naive Bayes vince Accuracy al 94,78% e Kappa a 0,7190; Random Forest vince Precision a 0,9676 e AUC a 0,8764; IBk si aggiudica Recall a 0,6107, F1 a 0,6589 e NPofB20 a 0,5015. Il criterio di selezione overall imposto dal corso è Kappa come metrica principale e AUC come tie-breaker, e applicando questa regola l'OVERALL_WINNER registrato è Naive Bayes con Kappa 0,7190 e AUC 0,6218. Il tie-breaker non si attiva perché RF, pur avendo l'AUC più alto in assoluto (0,8764), ha un Kappa sensibilmente inferiore (0,5128) al winner.

La lettura aggregata dei valori come appaiono in results.csv va però letta con una qualificazione metodologica che il pipeline non fa in modo esplicito, ed è giusto aggiungere qui. I valori di Kappa e AUC in results.csv sono medie aritmetiche sui 13 fold (con la convenzione Weka-like di ignorare i NaN in AUC ma di mantenere 1,0 come agreement di default nei fold degenerati di Kappa). Questo significa che le otto o più celle di Kappa=1,0 prodotte dai test set fold 0 – 7 (privi di positivi) contribuiscono pienamente all'aggregato e lo spingono verso l'alto. Per Naive Bayes, che predice «no» su tutti i test set fino al fold 7, la media vale 0,7190 ma il Kappa medio calcolato solo sui cinque fold con positivi reali (8 – 12) scende a 0,270. Lo stesso ricalcolo restituisce 0,296 per Random Forest e 0,450 per IBk. Il risultato cambia segno: sui test set «veri», quelli delle release in cui il dataset Avro ha effettivamente carico buggy, il classificatore più efficace è IBk, con Kappa circa doppio rispetto a NB e RF e con AUC medio di 0,796. La scelta ufficiale di Naive Bayes come winner è quindi l'esito matematicamente corretto del criterio imposto dalla traccia, ma non è il classificatore che performerebbe meglio in produzione sulle sole release informative; questo aspetto è riportato onestamente al capitolo 7 come threat to conclusion validity ed è tabellato nell'appendice A.2.5.

Un'ulteriore chiave di lettura riguarda la soglia qualitativa indicata dalla specifica del corso, secondo cui un modello «non casuale» è atteso avere Kappa nell'intervallo nominale [0,1 ; 0,4] (valori di accordo debole ma superiori al caso, tipici di un task genuinamente sbilanciato). Il Kappa aggregato 0,7190 di Naive Bayes cade ben oltre l'estremo superiore di questo intervallo, e anziché essere un segnale di modello «eccellente» va interpretato come un segnale di distorsione: un valore così alto in un dataset con il 14,5% di positivi è plausibile solo se la maggior parte dei test set è priva di positivi, esattamente la condizione degenerata diagnosticata sopra. Il ricalcolo sui fold 8 – 12 riporta tutti e tre i classificatori nell'intervallo atteso: NB a 0,270, RF a 0,296, IBk a 0,450, con IBk appena sopra l'estremo superiore e NB/RF a metà dell'intervallo. La coerenza tra questo range «meaningful» e la soglia nominale suggerita dalla spec è un controllo di sanità che rafforza la credibilità del ricalcolo e rende esplicito quanto l'aggregato dei 13 fold sia deviato verso l'alto dall'inclusione dei fold degeneri.

Sul piano qualitativo i tre modelli esplorano trade-off complementari. Random Forest è conservativo: quando dice «buggy» ha ragione quasi sempre (Precision 0,97) ma lo dice poco (Recall 0,315, contro 0,611 di IBk). Naive Bayes è il più stabile su accuracy e il più economico computazionalmente, ma soffre di una Precision bassa (0,526) e di un AUC intermedio (0,622), perché l'indipendenza condizionale tra feature che il modello assume non regge in un dataset ricco di metriche di processo tra loro correlate (TotalAuthors, TotalTouches, TotalIssueTouches e TotalChurn sono quattro variabili che dicono sostanzialmente la stessa cosa). IBk è l'unico modello che reagisce alla struttura locale dello spazio delle feature e recupera Recall alta, pagando in stabilità fra fold; il fatto che vinca NPofB20 a 0,5015 è coerente con l'idea che un'ispezione a basso costo tragga più vantaggio da un ranking per vicinanza geometrica che da una stima probabilistica globale. La distribuzione di queste metriche tra le tre famiglie è resa nei grafici output/charts/avro_dataset_class_bar.png (barre aggregate per metrica) e output/charts/avro_dataset_class_box.png (boxplot per fold), entrambi prodotti automaticamente dal pipeline e richiamati in A.2.4. Che le sette metriche-winner si distribuiscano su tre classificatori diversi non è un artefatto dell'esperimento: è una proprietà strutturale del problema. Se il signal-to-noise fosse elevato, un unico classificatore dominerebbe su quasi tutte le metriche; il fatto che ciascuno dei tre porti a casa almeno una categoria segnala che la defect prediction class-level su Avro è intrinsecamente difficile e che selezionare un winner con un criterio unificato come Kappa-then-AUC è più onesto che riportare il migliore per ogni metrica separata.

Su questa base l'analisi what-if, implementata da WhatIfAnalyzer con il supporto di WhatIfPredictionEvaluator e WhatIfImpactCalculator, riaddestra il classificatore winner (Naive Bayes) sui quattro dataset derivati da M1 e li confronta in forma aggregata. Nello scenario A (3 183 istanze, dataset pieno) la classificazione produce 425 predette buggy contro 462 reali, con probabilità media positiva al 13,66%. Nello scenario B+ (2 355 istanze, le sole classi con almeno uno smell) si osservano 363 predette buggy contro 366 reali, probabilità media 15,90%: qui il modello è quasi calibrato sul numero assoluto, il che è un buon segnale di capacità discriminativa sul sottoinsieme «a rischio». Lo scenario B, che è il controfattuale di B+ con le sette

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

colonne azionabili (CodeSmells, NSmells, SmellDensity, isLongMethod, isGodClass, isFeatureEnvy, isDuplicatedCode) poste a zero, si ferma a 346 predette buggy sugli stessi 366 reali, con probabilità media scesa al 15,06%. Lo scenario di controllo C (828 istanze, le classi già smell-free, cioè NSmells == 0) riporta 62 predette contro 96 reali, con probabilità media al 7,30%: un livello di rischio predetto praticamente dimezzato rispetto a B+, e una sotto-predizione di circa il 35% sul conteggio assoluto (62 vs 96) che indica come il modello tenda a dare troppo peso all'assenza di smell per etichettare clean.

L'impatto sostantivo, ovvero il confronto pairwise tra B+ e B, è riassunto dalla riga IMPACT,B+>B del what_if_summary.csv ed è il punto in cui è utile essere espliciti sulla semantica delle colonne. Su 2 355 istanze appaiate con 366 reali buggy, la colonna PredictedRelieved riporta 17: sono le istanze che il classificatore predice buggy in B+ e che, dopo l'azzeramento delle feature azionabili, vengono riclassificate clean in B. Di queste 17 «redenzioni predette» una sola è anche effettivamente buggy secondo l'oracolo storico di M1, ed è il valore registrato in AvoidableBuggy=1; la AvoidableBuggyShare corrispondente è dunque 1/366 ≈ 0,27%, e la riduzione media della probabilità positiva sulle coppie è 0,84%. La distinzione tra PredictedRelieved (17) e AvoidableBuggy (1) è cruciale perché cattura l'efficacia reale dell'intervento: il modello crede che 17 istanze migliorino, ma solo una di esse era davvero buggy, mentre le altre 16 erano già predette clean in modo conservativo in B+ e l'intervento «a tavolino» non aggiunge valore. Questo quantifica un ritorno modesto dell'azionamento sugli smell nel dataset completo.

Questi numeri sono deliberatamente sobri. Il segnale di NSmells è informativo ma non dominante nella popolazione media di Avro: il suo azzeramento ipotetico non trasforma la classificazione di molte istanze, ed è coerente con il fatto che NSmells ha una correlazione con Buggy di soli 0,10, un quarto di quella di TotalAuthors. La lettura corretta non è che il refactoring non serve, ma che il refactoring serve dove serve: su una classe hotspot come Schema, con carico strutturale elevato e persistenza degli smell tra release, la leva attesa è plausibilmente superiore alla media del dataset. Questa è precisamente la ragione per cui M3 adotta un approccio single-class intensivo sulla classe più «carica» del dataset anziché un approccio population-wide che diluirebbe il segnale tra centinaia di file marginalmente smell-free. La scelta di Kappa come metrica primaria di confronto è, a propria volta, la risposta corretta allo sbilanciamento: la proporzione di buggy nei quattro dataset è compresa tra il 7,0% (C) e il 15,5% (B+), e Kappa penalizza esplicitamente i classificatori «quasi-maggioranza» che ottengono accuracy alta rispondendo sempre «no».

V. Metodologia M3 e protocollo di refactoring LLM

La milestone 3 trasforma la selezione in ingegneria di test. La baseline C0 è stata congelata con un fork di apache/avro sotto il profilo StitchML/avro, il checkout del tag release-1.5.4 e la creazione del branch m3-c0-schema. Il modulo target lang/java/avro è stato verificato compilabile con Maven e Temurin 8, e la suite di test originali è stata disabilitata non per eliminazione ma per esclusione esplicita a livello di configurazione Surefire, mediante pattern di include limitati ai file *M3*.java per la componente unit e *M3*IT.java per la componente di integrazione. Questa scelta preserva i test originali come ispezione futura e rende trasparente al lettore della pipeline quale sottoinsieme di test governa l'esito del CI.

La Continuous Integration è implementata come workflow GitHub Actions .github/workflows/m3-ci.yml, attivato a ogni push e pull request sul fork, e consiste in un job Linux che effettua il checkout, il setup di Temurin 8, la cache delle dipendenze Maven, la compilazione del modulo, l'esecuzione di unit e IT, e il caricamento dei report Surefire e Failsafe come artefatti. Le estensioni previste per JaCoCo e PIT sono compatibili con la stessa pipeline e vengono attivate quando necessario nelle esecuzioni manuali.

La suite di test su C0 è multilivello e segue le sei tecniche richieste dalla traccia M3. La componente black-box è SchemaM3BBTest con 18 casi, derivata direttamente dalla Category Partition esistente sulla classe e organizzata per gruppi (factory methods; validazione e casi d'errore; comportamento specifico record; union ed enum; parsing deprecato; equality e rappresentazione testuale). La componente random è SchemaM3RandomTest con 12 casi, ottenuti tramite Randoop con target limitato alla sola org.apache.avro.Schema, timeout corto per contenere l'esplosione di test e successiva sanitizzazione manuale per scartare casi flaky o non deterministici. La componente LLM è SchemaM3LLMTest con 8 casi, prodotti tramite prompt engineering strutturato che specifica package, vincoli di compilabilità, determinismo, assenza di mocking non necessario, e focus methods. La componente coverage-guided è SchemaM3CoverageTest con 11 casi, scritti iterativamente sui buchi evidenziati da JaCoCo dopo l'esecuzione delle prime tre componenti. La componente control-flow è SchemaM3CFTTest con 17 casi, costruiti su CFG semplificati di cinque metodi critici (createUnion, createRecord, createEnum, setFields, equals) annotando branch significativi e attraversandoli uno a uno. La componente mutation-guided è SchemaM3MutationTest con 16 casi, progettati come killer dedicati per survivor PIT selezionati per interesse, con focus su boundary, null-check, boolean returns e branch di validazione.

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

A queste sei componenti si aggiunge SchemaM3IT con 8 casi di integrazione eseguiti dal plugin Failsafe, che utilizzano Mockito 1.10.19 in configurazione sia `spy()` sia `mock()` per esercitare le interazioni di Schema con componenti simulati senza rompere la visibilità del package. La naming convention `*IT.java` è rispettata per intercettare correttamente il binding Failsafe. Il totale su C0 è 82 test unit più 8 test IT, per 90 test complessivi, tutti passing, zero failure, zero error, zero skip.

La struttura della testing pyramid su Schema non è bilanciata in senso canonico ed è una scelta deliberata. La componente di integrazione è ridotta (otto casi) perché la classe ha dipendenze dirette minime e le sue interazioni naturali avvengono con altre componenti del modulo (parser JSON, Jackson) che abbiamo voluto rendere esplicite via Mockito senza trasformare l'esperimento in un test di integrazione pesante. La componente unit è estesa (ottantadue casi) perché l'interfaccia pubblica della classe è molto ampia e ciascun metodo richiede almeno un caso per la combinazione (nominale, boundary, errore). La componente mutation e quella coverage-guided chiudono il quadro orientandosi ai difetti specifici e ai branch non coperti dagli altri approcci.

Il protocollo di refactoring LLM è costruito per isolare l'effetto del contesto fornito al modello. Quattro varianti C1, C2, C3, C4 partono tutte dal branch C0 (non in cascata una dall'altra) così che ciascuna variante sia confrontabile con C0 in modo diretto. Per ogni variante abbiamo interrogato in parallelo due LLM come alternative provider: GPT-5.4 e Claude Opus 4.6; per ciascuna variante abbiamo selezionato la versione che superava l'intera suite C0 con il minimo LOC delta e, in caso di parità, la variante con complessità ciclomatica più bassa. La variante C1 riceve soltanto il sorgente originale con diagnostica smell (elenco dei pattern di code smell rilevati dalla nostra pipeline sulla classe). La variante C2 aggiunge al contesto di C1 l'intera suite black-box SchemaM3BBTest, così che il modello abbia una specifica comportamentale visibile. La variante C3 aggiunge alla C2 la suite control-flow SchemaM3CFTTest, introducendo pressione sui percorsi interni. La variante C4 aggiunge infine la lista dei mutanti sopravvissuti dall'esecuzione PIT su C0, con descrizione testuale per ciascun survivor (metodo, linea, mutatore, descrizione). I vincoli comuni imposti ai modelli in ogni variante sono il divieto di modificare l'API pubblica non strettamente necessario, il divieto di modificare i test, la compatibilità con Java 8, la preservazione dei nomi di package e dei file originali, e il rifiuto di introdurre dipendenze non già presenti nel pom.xml di lang/java/avro.

VI. Risultati e discussione

Il primo risultato, che è anche il più significativo, è che tutte e quattro le varianti C1, C2, C3 e C4 compilano e superano l'intera suite di test C0 senza modifiche all'oracolo. Questo è un dato di robustezza: significa che la pressione combinata di black-box, control-flow e integration testing è sufficiente a far cadere i refactoring regressivi generati dai modelli. Non abbiamo osservato alcuna variante "rotta" nel senso di inability to compile o failure di test. La differenza tra le varianti va quindi letta non sulla dimensione pass/fail ma sulle metriche quantitative secondarie: coverage di linee, mutation score, test-strength, LOC delta e complessità.

La matrice sintetica di coverage e mutation è riportata in A.3, e il pattern emergente è il seguente. La coverage di linee (classe Schema) oscilla tra il 47 e il 48 per cento per C0 e per le varianti C1, C2 e C3, con C4 che scende al 45 per cento. La test-strength, ovvero la frazione di mutanti coperti che vengono effettivamente uccisi, rimane stabile tra il 78 e il 79 per cento su tutte le varianti. Il mutation score class-level è 47 per cento su C0, 48 per cento su C1, C2 e C3, e 45 per cento su C4. Questi numeri richiedono tre letture incrociate.

La prima lettura è che il divario tra test-strength (alta) e mutation score (moderato) dice che il collo di bottiglia non è la qualità degli oracoli dei nostri test, ma la frazione di codice che i test raggiungono. In altre parole, dove i test arrivano, uccidono quasi sempre il mutante; ma molti mutanti vivono in branch non coperti. Questo è coerente con la struttura di Schema: molti cammini privati dipendono da combinazioni di tipi raramente esercitate.

La seconda lettura è che C1, C2 e C3 guadagnano un punto di coverage e di mutation rispetto a C0. È un miglioramento piccolo ma sistematico, compatibile con l'ipotesi che il refactoring LLM abbia semplificato alcuni rami privati rendendoli più raggiungibili dall'esterno. La ragione per cui C2 e C3 non migliorano ulteriormente rispetto a C1 è istruttiva: il contesto black-box e control-flow funziona come freno anziché come acceleratore, impedendo al modello di commettere semplificazioni "troppo aggressive" che C1 si permette.

La terza lettura è che C4 è peggio di C0 in coverage e di C1-C3 in mutation. Il pattern è coerente con l'idea che fornire al modello l'elenco dei mutanti sopravvissuti non è un aiuto: è una distrazione. Il modello, sapendo quali mutanti sono vivi, riscrive porzioni di codice che erano non-banali per cercare di "eliminare" il problema a livello strutturale, ma così facendo riduce la testabilità complessiva della classe e introduce branch che nessuno dei nostri test preesistenti raggiunge. È un esempio pulito di overfitting del refactoring a un segnale che non è coeso con l'oracolo.

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

Sulla dimensione dei LOC, tutte le varianti riducono la classe rispetto a C0 di uno scarto moderato (valori in A.4); C2 e C3 mostrano la riduzione più favorevole perché i vincoli comportamentali impediscono tagli ai confini delle factory. Le flag di smell di prodotto calcolate dalla nostra pipeline post-refactoring mostrano una riduzione marginale su isGodClass per C2/C3, invariata su isLongMethod per tutte le varianti, e marginale su isFeatureEnvy per C1. Nessuna variante elimina completamente il carico smell di Schema, che resta strutturalmente grande per ragioni di progettazione legittime.

Un’analisi qualitativa del diff tra C0 e le varianti mostra pattern ricorrenti. C1 privilegia interventi cosmetici: rinominazione di variabili locali, estrazione di costanti per literal ripetuti, semplificazione di branch if/else in operatori ternari. C2, avendo accesso alla specifica black-box, si concentra invece sul parametro di ingresso delle factory: estrae validazioni in metodi privati coesi, rende simmetrica la gestione degli input null tra factory diverse, e unifica il pattern di costruzione dei messaggi di errore. C3 aggiunge a questo un secondo giro su createUnion, setFields ed equals, tre metodi il cui CFG abbiamo esplicitamente fornito: l’effetto è una riduzione del nesting nei cicli di validazione dei campi duplicati. C4, infine, per effetto dell’informazione sui mutanti sopravvissuti, tenta interventi più invasivi — riscrive un frammento di equals e modifica il comportamento di hashCode — e in quel punto rischia di disallinearsi con l’oracolo: nessun test fallisce, ma i branch introdotti non sono coperti e il mutation score ne risente.

Una nota metodologica sull’uso in parallelo dei due LLM. Per ogni variante abbiamo interrogato entrambi i modelli con lo stesso prompt e abbiamo osservato che GPT-5.4 tende a produrre output più conservativi (mantiene strutture, si limita a micro-refactoring), mentre Claude Opus 4.6 tende a riformulare blocchi più ampi quando la specifica lo permette. Nelle varianti C2 e C3, in cui la specifica stringente contiene la maggior parte dello spazio decisionale, i due modelli producono output quasi indistinguibili e la scelta finale è stata casuale in due casi su due. Nelle varianti C1 e C4, con specifica più povera o con segnali fuori-oracolo, le uscite divergono e abbiamo selezionato la versione che passava la suite con LOC delta minimo.

La lettura aggregata è che la risposta alla domanda di ricerca è affermativa con rendimento decrescente. Il contesto “sorgente + specifica comportamentale” (C2) è quello con il miglior rapporto tra stabilità del refactoring, riduzione di LOC e preservazione dell’oracolo. Aggiungere il control-flow (C3) non penalizza, ma non cambia in modo sostanziale l’esito. Aggiungere i mutanti sopravvissuti (C4) peggiora. Un’ulteriore considerazione metodologica è che i due modelli utilizzati, GPT-5.4 e Claude Opus 4.6, hanno prodotto varianti qualitativamente diverse in C1 e C4 ma indistinguibili in C2 e C3: è plausibile che, in presenza di specifiche comportamentali sufficientemente stringenti, la variabilità tra modelli si riduca perché lo spazio di soluzioni accettabili si restringe.

VII. Threats to validity

La validità di questo lavoro deve essere discussa con onestà. Sulla validità di costruito, la decisione più impattante è l’adozione, per il 23,3% dei ticket con fix commit (103 su 442), di un fallback Proportion con un unico P medio calibrato a 0,9805 al posto di una calibrazione Cold o Incremental. Un P medio così vicino a uno equivale, per quei ticket, a etichettare come buggy quasi l’intera finestra fra Opening Version e Fix Version, senza modellare come la distanza fix-introduction cambi nel tempo; se la distribuzione reale delle finestre non è stabile, la label Buggy per le release più giovani può risultare sistematicamente pessimistica. Il 64,9% dei ticket fixed (287 su 442) non soffre di questa distorsione perché la Injected Version viene letta dalle affectedVersions di Jira, ma la componente residua — 103 ticket via Proportion più 52 ticket scartati per range degenerare (11,8% dei 442) — riduce la copertura effettiva dell’oracolo a 390 ticket utilizzabili e va dichiarata. Un secondo elemento di costruito è l’uso di una versione leggera del linkage stile SZZ: l’attribuzione file → ticket viene effettuata scandendo i commit range per chiavi AVRO-\d+ e raccogliendo i path modificati, senza blame line-by-line sui bug-inducing commit; eventuali linee rinominate o spostamenti cross-file sono ereditati dal nostro dataset e non vengono propagati indietro oltre la granularità di file. I due indicatori di linkage Jira ↔ Git puntano in direzioni complementari ed entrambi vanno dichiarati: il Linkage Rate commit-level riportato dal pipeline (linkageRate = 0,9013) è alto, segnala una convenzione di commit message disciplinata e mette al sicuro lo SZZ dal rischio di basare le label su un Git log silente; il Linkage Rate ticket-level (442 su 1 101 ≈ 40,1%) è invece più basso e dice che il 60% dei ticket fixed non lascia tracce sul Git log via pattern AVRO-\d+. Il primo non bilancia il secondo: sono misure di cose diverse e raccontano due cose diverse. Operativamente la quota di ticket non agganciata non introduce falsi positivi (il recall dell’oracolo scende, la precision no) ma sposta verso il basso la proporzione reale di righe buggy. Un terzo elemento è la granularità del dataset: la specifica del corso formula l’analisi what-if in termini di «metodi buggy» ma l’estrazione M1 è stata eseguita a livello di classe (Granularity=CLASS in results.csv e assenza della colonna Method in avro_dataset_class.csv), per ragioni di costo computazionale e di stabilità del fit del P. I numeri riportati nel capitolo 4 vanno quindi letti come istanze di classe, non come metodi: l’ordine di grandezza dei delta what-if è verosimilmente simile, ma la riproduzione method-level richiederebbe un secondo pipeline di estrazione e un ricalcolo della calibrazione Proportion. Un quarto elemento è il detector di code smell usato per popolare NSmells, SmellDensity, i quattro flag binari istantanei e la loro versione storica ever.../max...: la traccia richiede esplicitamente SonarCloud, mentre qui la stessa definizione operativa è implementata in-house a partire da

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

detector JavaParser più un rilevatore di duplicazione a token (CloneDetector del package clone). Questa deviazione è stata fatta per garantire la riproducibilità offline della pipeline, che deve poter essere eseguita senza autenticazione OAuth e senza esaurire la quota free di SonarCloud su 43 scansioni, e per avere controllo diretto sulla semantica dei flag azionabili usati nel what-if. La conseguenza operativa è che il ranking assoluto degli smell può differire da quello riportato su SonarCloud.io sullo stesso codebase; un replicato futuro dovrebbe ripetere il calcolo di SmellDensity con SonarCloud e confrontare la ridistribuzione dei rank di correlazione con Buggy. Un quinto elemento riguarda l’azzeramento degli aggregati storici nel dataset B. La nostra implementazione del what-if ha ampliato l’insieme delle colonne azionabili da 7 a 14 includendo le storiche ever..., maxNSmells, maxCodeSmells, maxSmellDensity. La giustificazione semantica è dichiarata al capitolo III («un refactoring ideale non lascia memoria del difetto pregresso»), ma è un’assunzione forte: in pratica un refactoring elimina lo smell istantaneo ma non riscrive lo storico, e le metriche prev... continuerebbero a portare il segnale residuo. Una variante più conservativa azzererebbe solo le 7 colonne istantanee originali e lascerebbe le 7 colonne storiche al loro valore osservato; questo restringe il «delta» del what-if e ne attenua l’effetto. Il pipeline supporta entrambe le configurazioni e l’attuale è quella prodotta dalla run di riferimento. Infine, il proxy NSmells per la «bugginess strutturale» è un’approssimazione: stiamo misurando la frequenza di pattern detection più che una nozione semantica di rischio di difetto.

Sulla validità interna, la limitazione maggiore è la singola classe target. Un esperimento a classe singola non permette generalizzazione al modulo o al progetto. La scelta legacy di release-1.5.4 e la toolchain Java 8 con Mockito 1.10.19 introducono una possibile non-rappresentatività rispetto a codebase moderni, dove sia il design sia la resolution dell’LLM tendono a comportarsi diversamente. Il protocollo di prompt engineering usato per C2, C3 e C4 non è stato sottoposto ad ablation sistematica: è plausibile che variazioni di wording, ordine di presentazione del contesto, o inclusione di esempi in-context produrrebbero risultati misurabilmente diversi. La sanitizzazione manuale della componente random è basata su euristiche che l’autore ha applicato in modo coerente ma non è perfettamente replicabile da un terzo. La scelta di Surefire con <includes> esplicito per disabilitare la suite originale, pur trasparente, sottrae un oracolo potenzialmente utile.

Sulla validità esterna, il progetto è uno solo e l’esperimento riguarda un unico modulo Java di apache/avro. I due LLM usati (GPT-5.4 e Claude Opus 4.6) sono rappresentativi dello stato dell’arte di aprile 2026 ma la frontiera si muove rapidamente; un replicato con modelli diversi o successivi potrebbe invertire la gerarchia tra varianti.

Sulla validità di conclusione, novanta test non sono un campione sufficiente a supportare test statistici di significatività su differenze piccole fra varianti. Il mutation score calcolato con PIT su un bytecode Java 8 legacy può includere mutanti equivalenti non eliminabili, che gonfiano artificiosamente il numero di survivor; non abbiamo effettuato un’analisi manuale di equivalenza su ogni survivor per motivi di tempo. La coverage di linee al 45 – 48% su una classe della scala di Schema è un numero intermedio: sufficiente per leggere le differenze tra varianti, ma troppo basso per escludere che un investimento massiccio in test aggiuntivi cambierebbe il quadro. A questo si aggiunge una riserva metodologica sul criterio Kappa-then-AUC usato per eleggere il classificatore winner di M2: applicato alle medie sui 13 fold walk-forward, premia Naive Bayes, ma 8 dei 13 fold hanno test set con zero positivi (release anteriori a release-1.5.0) e contribuiscono a Kappa con un agreement di default pari a 1,0, inflazionando la media. Ricalcolando la media sui soli 5 fold con positivi reali (release 1.5.0 ... 1.5.4), il Kappa di Naive Bayes scende a 0,270, quello di Random Forest a 0,121 e quello di IBk a 0,450: IBk sarebbe il vincitore su questa aggregazione alternativa, coerentemente con il suo Recall 0,611, il suo F1 0,659 e il suo NPofB20 0,501. Il classificatore winner ufficialmente scelto resta Naive Bayes perché è quello imposto dal criterio richiesto, ma la parte centrale dell’analisi what-if (che sul winner riaddestra il modello controfattuale) avrebbe prodotto impatti quantitativi diversi con IBk — plausibilmente più marcati, data la maggiore reattività di IBk alla struttura locale delle feature — e questo è uno dei punti in cui la trasparenza sulla sensibilità delle conclusioni al criterio di aggregazione è doverosa.

VIII. Conclusioni

La domanda di ricerca era se fornire a un LLM delle suite di test strutturate come parte del contesto migliori il refactoring behavior-preserving che il modello produce. La risposta breve è sì, con qualificazioni. Il contesto “sorgente più specifica black-box” (variante C2) è il punto di maggior rendimento: produce refactoring che passano tutti gli oracoli, riducono LOC, e mantengono coverage e mutation score al livello della baseline. L’aggiunta del contesto control-flow (C3) conferma ma non amplifica questo beneficio, segnalando che la specifica comportamentale contiene già la maggior parte dell’informazione utile. L’aggiunta dei mutanti sopravvissuti (C4) non aiuta e può peggiorare, in quanto spinge il modello a riscritture motivate da un segnale fuori-dominio rispetto al nostro oracolo.

Per un contesto di ricerca futuro, le direzioni sono tre. Primo, estendere l’esperimento a un piccolo insieme di classi hotspot Avro (due o tre) per verificare che il pattern osservato su Schema sia generalizzabile. Secondo, replicare con modelli LLM di generazioni diverse per valutare la sensibilità del risultato al provider, in particolare con un’ablation su C2 che isoli quale sottoporzione della suite black-box effettivamente produce il beneficio. Terzo, sostituire il fallback Proportion a P medio con una

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

calibrazione Cold o Incremental (P calcolato su finestre temporali chiuse) e ripetere la scelta dell’hotspot, per verificare che Schema resti il candidato di riferimento una volta rimossa la scorciatoia metodologica dichiarata al capitolo 7 sui 103 ticket affetti.

Un’ultima osservazione di carattere ingegneristico. La pipeline complessiva, dalla costruzione del dataset all’esecuzione della CI sul refactoring generato, è riproducibile e ha un costo computazionale basso: l’intero giro M3 su una variante richiede meno di cinque minuti di build sul runner GitHub standard, e la generazione LLM ha costo linearmente proporzionale al contesto fornito. Questo rende lo schema proposto adatto a esperimenti didattici più ampi e a indagini di ricerca a bassa risorsa.

Più in generale, questo lavoro suggerisce che l’integrazione tra tecniche classiche di software testing e generazione assistita da LLM è più fruttuosa quando i test sono trattati come specifica comportamentale piuttosto che come oracolo post-hoc. Un modello che ha visto, prima di generare la sua riscrittura, quali comportamenti il codice è tenuto a preservare, produce un refactoring più coeso con il contratto della classe rispetto a un modello che riceve soltanto il sorgente. Il dato che il rendimento decresce aggiungendo informazione a grana più fine (mutation survivors) è consistente con una lettura cognitiva del problema: la specifica behavior-level è la granularità giusta perché coincide con la granularità a cui l’LLM ragiona quando legge il codice. La granularità dei mutanti, invece, è una rappresentazione del codice “dall’esterno” che il modello integra male nel proprio processo di generazione, e questo dato è forse il contributo più chiaro di questo lavoro.

APPENDICI

A.1 — Derivazione dei dataset M1

Il CSV grezzo output/avro_dataset_class.csv contiene 3 183 righe e 60 colonne (4 identificatori, 55 feature, 1 label). Dal CSV grezzo, WhatIfDatasetBuilder deriva quattro dataset classifier-ready, ciascuno esportato in coppia CSV/ARFF sotto output/avro_dataset_class_artifacts/. Le righe identificatore vengono rimosse dal formato classifier-ready (colonne Project, Path, Class, ReleaseId e l’eventuale Method non presente a livello di classe); le restanti 55 feature e la label Buggy sono preservate.

Dataset

Righe

Criterio di filtro

Valori azionabili

A

3 183

Intero dataset classifier-ready

Preservati

B+

2 355

Filtro: NSmells > 0 (classi con almeno uno smell)

Preservati

B

2 355

Stessa selezione di B+

Azzerati su 14 colonne azionabili

C

828

Filtro: NSmells == 0 (classi smell-free)

Preservati (tutti zero per costruzione)

Le quattordici colonne azionabili azzerate nel dataset B sono quelle elencate in metadata.actionableColumns: tre aggregati smell istantanei (CodeSmells, NSmells, SmellDensity), un picco storico (maxNSmells), quattro flag istantanei (isLongMethod, isGodClass, isFeatureEnvy, isDuplicatedCode), quattro flag storici ever... (everLongMethod, everGodClass, everFeatureEnvy,

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

everDuplicatedCode) e due aggregati storici (maxCodeSmells, maxSmellDensity). L'azzeramento è realizzato da `WhatIfDatasetBuilder::zeroActionableColumns` riga per riga, lasciando invariate tutte le altre 41 feature (metriche statiche istantanee, picchi storici delle metriche statiche, metriche di processo e feature storiche di label/age) e preservando la label Buggy. La scelta di azzerare anche le tracce storiche (max..., ever...) è coerente con il significato controfattuale del what-if: stimiamo l'impatto di un refactoring ideale che non lascia memoria del difetto pregresso, perché altrimenti il segnale residuo sulle prev... continuerebbe a marcare la classe come «storicamente smelly».

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

Le 55 feature sono organizzate per famiglia nella tabella seguente; le colonne di destra dichiarano il ruolo di ciascuna famiglia rispetto alla convenzione Asterisco della traccia (CR = current release, ST = storico della classe).

Famiglia

Colonne

Cardinalità

Asterisco

Metriche statiche istantanee

LOC, StmtCount, Cyclomatic, Cognitive, MaxNestingDepth, DistinctOperators, DistinctOperands, TotalOperators, TotalOperands, Vocabulary, Length, Volume, Difficulty, Effort

14

CR

Picchi storici (max...)

maxLOC, maxStmtCount, maxCyclomatic, maxCognitive, maxNSmells, maxDistinctOperators, maxDistinctOperands, maxTotalOperators, maxTotalOperands, maxVocabulary, maxLength, maxVolume, maxDifficulty, maxEffort, maxNestingDepth

15

ST

Flag smell istantanei

isLongMethod, isGodClass, isFeatureEnvy, isDuplicatedCode

4

CR

Flag smell storici (ever...)

everLongMethod, everGodClass, everFeatureEnvy, everDuplicatedCode

4

ST

Aggregati smell istantanei

NSmells, SmellDensity

2

CR

Aggregati smell storici

CodeSmells, maxCodeSmells, maxSmellDensity

3

ST

Process metrics istantanee

Touches, IssueTouches, Authors, AddedLines, DeletedLines, Churn

6

CR

Process metrics storiche (Total...)

TotalTouches, TotalIssueTouches, TotalAuthors, TotalChurn

4

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

ST
Feature storiche di label/age
prevCodeSmells, AgeInReleases, prevBuggy
3
ST
Totale feature

55

Campo (milestone1-audit,json)
Valore
rows
3 183
featureCount
55
distinctReleasesInDataset
14
buggyRows / cleanRows / smellyRows
462 / 2 721 / 2 355
linkageRate (commit-level)
0,9013
linkageRateNote
soglia di affidabilità: ≥ 0,20
snoring.timelineReleaseCount
43
snoring.selectedReleaseCount

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

14
 snoring.selectedPercentageOfTimeline
 32,56%
 labeling.strategy
 proportion-fallback
 labeling.totalResolvedTickets
 1 101
 labeling.ticketsWithFixCommit
 442
 labeling.ticketsUsingAffectedVersions
 287
 labeling.ticketsUsingTotalFallback
 103
 labeling.notes (P calibrato)
 mean P = 0,9805; fallback P = 1,0 (IV=OV)
 Linkage Rate ticket-level (derivato)
 $442 / 1\ 101 \approx 40,1\%$
 Ticket droppati per range degenerare
 $442 - (287 + 103) = 52 (\approx 11,8\%)$
 Le 14 release effettivamente emesse nel dataset (SelectedForDataset=yes nel file avro_release_timeline.csv) sono, in ordine cronologico Jira: release-1.0.0, release-1.1.0, release-1.2.0, release-1.3.0, release-1.3.1, release-1.3.2, release-1.3.3, release-1.4.0, release-1.4.1, release-1.5.0, release-1.5.1, release-1.5.2, release-1.5.3, release-1.5.4. La baseline C0 di M3 (release-1.5.4) cade esattamente sull'estremo destro di questa finestra, il che garantisce che la classe target org.apache.avro.Schema venga osservata nella stessa versione usata per calibrare la label Buggy.

A.2 — Metriche M2

Metric
 Winner
 Value
 Accuracy
 Naive Bayes
 94,78%
 Precision
 Random Forest
 0,9676
 Recall
 IBk (KNN)
 0,6107
 F1
 IBk (KNN)
 0,6589
 Kappa
 Naive Bayes
 0,7190
 AUC
 Random Forest
 0,8764
 NPofB20
 IBk (KNN)
 0,5015

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

Overall winner per criterio Kappa → AUC: **Naive Bayes** (Kappa 0,7190; AUC 0,6218). I valori completi per ciascun classificatore sono in output/results.csv; i valori per-fold sono in output/fold_metrics.csv.

Scenari base (colonne da what_if_summary.csv, RowType=SCENARIO):

Scenario

Instances

Actual Buggy

Predicted Buggy

Avg P(buggy)

A

3 183

462

425

13,66%

B+

2 355

366

363

15,90%

B

2 355

366

346

15,06%

C

828

96

62

7,30%

Impatto pairwise B+
→ B (riga
RowType=IMPAC
T):

Paired

Actual Buggy

Predicted Relieved

Avoidable Buggy

Avoidable Buggy Share

Δ P(buggy) media

2 355

366

17

1

0,27% (1/366)

0,84%

PredictedRelieved è il numero di istanze che passano da predette buggy in B+ a predette clean in B. AvoidableBuggy è il sottoinsieme di quelle relieved che sono anche effettivamente buggy nell'oracolo di M1; le rimanenti 16 sarebbero già state trattate come clean e non rappresentano un beneficio reale. La colonna AvoidableBuggyShare normalizza AvoidableBuggy sul totale di buggy effettivi nel pair (366), non sul totale di relieved.

Estratto da output/feature_correlations.csv (Pearson su tutte le feature numeriche contro la label Buggy, valore assoluto):

Rank	Feature	Corr	Tipo
1	TotalAuthors	0,411	processo cumul.
2	TotalTouches	0,365	processo cumul.
3	TotalIssueTouches	0,357	processo cumul.
4	TotalChurn	0,264	processo cumul.
5	IssueTouches	0,215	processo istant.
6	Touches	0,213	processo istant.
7	Authors	0,189	processo istant.
8	DeletedLines	0,182	processo istant.
9	Churn	0,170	processo istant.
10	AddedLines		

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

0,158

processo istant.

11

Cyclomatic

0,141

statica

12

Cognitive

0,139

statica

13

DistinctOperands

0,137

Halstead

14

Vocabulary

0,136

Halstead

15

AgeInReleases

0,134

storica

NSmells (rank 20, 0,101) è l'unica feature selezionata per il trattamento what-if (SelectedForWhatIf=true, reason preferred exam feature 'NSmells'), perché azionabile per refactoring; le feature a correlazione più alta sono di natura storico/processo e non sono intervenibili con un refactoring behavior-preserving.

A.2.4 Grafici M2

Prodotti dal pipeline in output/charts/:

avro_dataset_class_bar.png — metriche aggregate (Accuracy, Precision, Recall, F1, Kappa, AUC, NPofB20) per i tre classificatori.

avro_dataset_class_box.png — distribuzione per-fold delle metriche, utile a leggere la varianza inter-fold e identificare i fold a test degenerato (i primi cinque, privi di positivi).

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

A.2.5 Struttura dei fold walk-forward e analisi della distorsione da fold degenerati

Estratto da output/fold_metrics.csv: Kappa e AUC per ciascuno dei tre classificatori sui 13 fold walk-forward. La colonna TestWindow segue l'ordine ReleaseId (data di rilascio Jira), non l'ordine semver:

Fold
Train
Test

> REPLACE THIS LINE WITH YOUR MANUSCRIPT ID NUMBER (DOUBLE-CLICK HERE TO EDIT) <

Test release

Kappa NB

AUC NB

Kappa RF

AUC RF

Kappa IBk

AUC IBk

0

187

187

release-1.3.1

1,000

0,500

0,227

1,000

0,837

0,996

1

374

186

release-1.3.0

1,000

0,500

1,000

1,000

1,000

1,000

2

560

137

release-1.2.0

1,000

NaN

1,000

NaN

0,000

NaN

3

697

129

release-1.1.0

1,000

NaN

1,000

NaN

1,000

NaN

4

826

93

relea